2.5 Object References and Missing Objects

An object variable identifies an object; it does not contain an independent copy of all the object's data. That distinction matters when assigning objects and when a requested object might not exist.

Two variables can identify the same object

```php
$person1 = new Person('Joe', '1990-01-01');
$person2 = $person1;

$person2->setName('Jane');

echo $person1->getName(); // Jane
echo $person1 === $person2 ? 'same object' : 'different objects';
```

Assignment copies the object identifier, so both variables point to the same object. Strict comparison with `===` confirms identical object identity. This differs from ordinary array assignment, which produces independent array values.

If an independent object is required, PHP provides `clone`, but cloning nested objects is a separate topic because the default clone is shallow.

Object relationships use references

```php
$instructor = new Person('Nora');
$course = new Course();
$course->setInstructor($instructor);
$instructor->setName('Nora Smith');
```

The course's instructor property and `$instructor` identify the same `Person`. If that person changes its display name, the course observes the same person with the updated state.

A method may return no object

A lookup can honestly return an object or `null`. Express that possibility in the return type:

```php
function findPerson(int $id): ?Person
{
    if ($id === 1) {
        return new Person('Nora', '1990-01-01');
    }

    return null;
}
```

```php
$member = findPerson(99);

if ($member === null) {
    echo 'Member not found.';
} else {
    echo $member->getDisplayName();
}
```

`?Person` is shorthand for `Person|null`. Returning `null` is not automatically an error; it is part of the method's contract and calling code must decide what it means.

The nullsafe operator

PHP's nullsafe operator `?->` stops a chain and evaluates to `null` when the expression on its left is `null`:

```php
$displayName = findPerson(99)?->getDisplayName();
echo $displayName ?? 'Member not found.';
```

Use `?->` when "no object" naturally means "no value." Use an explicit `if` when the missing case requires logging, a different response, or several statements. The nullsafe operator is read-only; it cannot assign through a possibly missing object.

Avoid the word-reference trap

PHP documentation often says object variables hold object identifiers. That is close to a reference for everyday explanation, but it is not the same as PHP's explicit reference operator `&`. You do not need `&` to share an object:

```php
$second = $first;  // both identify the same object
```